

# Classification d'images rétinienne par apprentissage profond

Deep Learning · Compétition Kaggle 2

---

Jegham Mohamed | Ben Mansour Louay

Polytechnique Sousse

**1 080**

images d'entraînement

**5**

classes de qualité

**4**

architectures comparées

**+2**

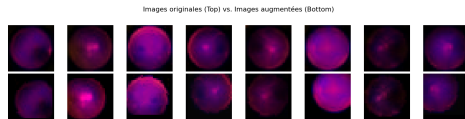
bonus Streamlit

**Objectif.** Estimer automatiquement la **qualité perceptive** d'une image rétinienne parmi **5 classes** ordonnées (0 = la plus basse  $\rightarrow$  4 = la plus haute).

**Jeu de données.**

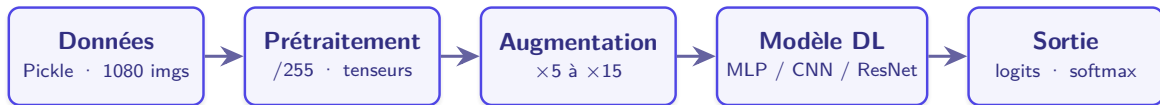
- **Entraînement** : 1 080 images RVB  $28 \times 28$
- **Test public (Kaggle)** : 400 images
- Classes *déséquilibrées*  $\rightarrow$  split **stratifié**
- *Petit jeu*  $\rightarrow$  risque **élevé de surapprentissage**

**Métriques.** Accuracy · **F1 macro** · matrice de confusion  
· **ROC** multiclasse (one-vs-rest, AUC macro).



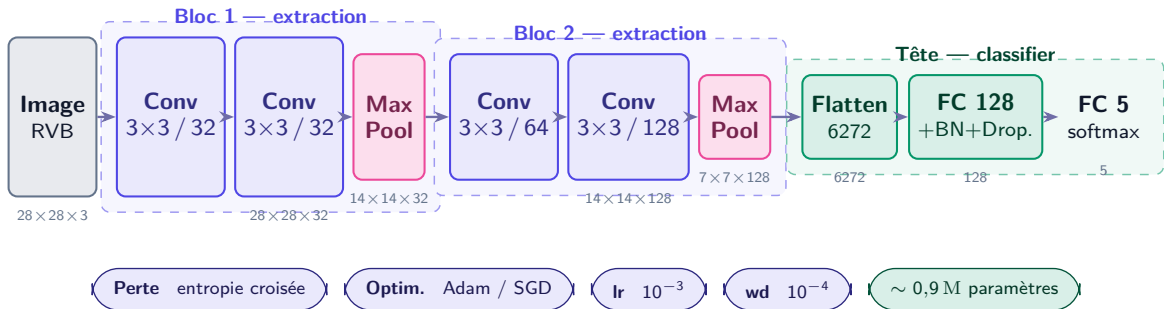
*Exemples originaux (haut) vs. augmentés (bas)*

# Pipeline Deep Learning



- ▶ **Augmentation** : flip, rotation, recadrage, ColorJitter, affine, equalize, sharpness.
- ▶ **Régularisation** : BatchNorm, Dropout, weight decay, **early stopping** sur la val. loss.
- ▶ **Optimisation** : Adam ou SGD + momentum, **ReduceLROnPlateau**, CrossEntropyLoss (label smoothing).
- ▶ **Validation** : découpage stratifié 80/20, meilleurs poids rechargés après entraînement.

# Architecture principale : cnn compacte



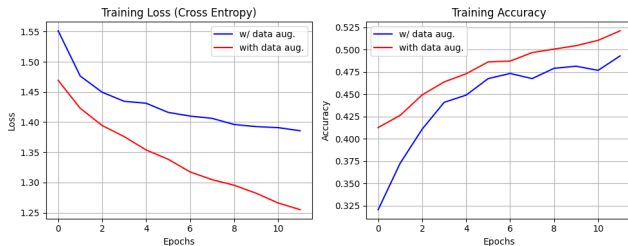
La hauteur des blocs reflète la résolution spatiale ; les groupes en pointillés mettent en évidence les trois étages de la CNN. Inspirée de [Nature](#) 41598-025-87171-9.

# Comparaison des architectures

| Modèle                  | # param.       | Train acc.  | Val acc.    | Test Kaggle |
|-------------------------|----------------|-------------|-------------|-------------|
| MLP baseline            | ~ 1,4 M        | 0,62        | 0,45        | —           |
| Kernel SVM (référence)  | —              | —           | —           | 0,46        |
| <b>cnn (production)</b> | ~ <b>0,9 M</b> | <b>0,53</b> | <b>0,51</b> | <b>0,52</b> |
| Small RESNET            | ~ 1,1 M        | 0,58        | 0,50        | —           |
| RESNET18 TL (ImageNet)  | ~ 11 M         | à compléter | à compléter | à compléter |

- ▶ **cnn** : meilleur compromis *précision* · *coût* · *surapprentissage* sur ce dataset.
- ▶ **mlp** : pas de prior spatial → gap train–val très marqué.
- ▶ **ResNet scratch** : skip connections utiles mais gains limités quand  $|\mathcal{D}|$  est faible.
- ▶ **resnet18 TL** : transfer learning sur ImageNet ; chiffres à reporter depuis `output/comparison_table.md`.

# Courbes d'apprentissage & surapprentissage

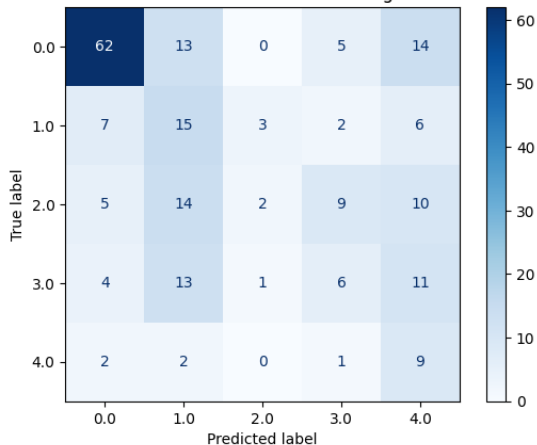


*Perte et accuracy — sans augmentation (bleu) vs. avec (rouge)*

- ▶ **Sans augmentation** : train chute, val. stagne  
→ **surapprentissage** clair.
- ▶ **Avec augmentation ( $\times 10$ )** : val. proche de train → meilleure généralisation.
- ▶ **Early stopping** déclenché vers les epochs 10–12 selon les runs.
- ▶ **BN + Dropout** stabilisent l'optimisation et réduisent la variance.

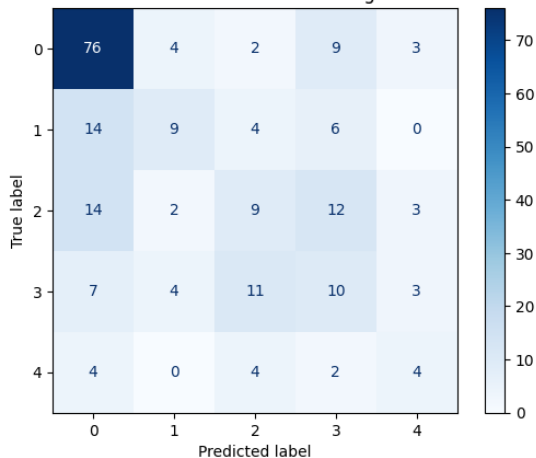
# Matrices de confusion

CNN Confusion Matrix without Data Augmentation



*Sans augmentation*

CNN Confusion Matrix with Data Augmentation



*Avec augmentation*

► Erreurs concentrées entre **classes adjacentes** (continuum de qualité).

# Conclusion & démonstration

## Livrables

- ▶ Pipeline reproductible (`train_and_save.py`) avec F1 / ROC et early stopping.
- ▶ **cnn** + augmentation comme modèle de référence ; **resnet18 TL** pour le transfer.
- ▶ App **Streamlit** : prédiction, lot, courbes, **Grad-CAM** (CNN / RESNET18).

## Limites & perspectives

- ▶ Très peu d'images → calibration (température), modèle pré-entraînés plus larges.
- ▶ **ViT** léger sur patches  $28 \times 28$  ; validation  $k$ -fold pour des IC.

### Démo Streamlit

Interface moderne · Plotly · cartes de confiance  
· Grad-CAM.

```
streamlit run app/streamlit_app.py
```

### Soumission Kaggle

Mettre à jour le tableau comparatif avec le **score public** après soumission du `.csv`.

**Merci de votre attention.**

Des questions ? (*MLP vs CNN · gradients · BN/Dropout · perte & optim.*)